

第4章 構文解析

4.1 構文解析の基礎

4.1.1 BNF 定義と CF 文法

プログラミング言語の構文は CF 文法 (文脈自由文法, context-free grammar) によって定義される。

定義 4.1. CF 文法 G は $G = \{N, \Sigma, P, S\}$ の四つ組で定義される。ここで、 N は非終端記号の集合、 Σ は終端記号の集合、 P は生成規則の集合、 $S \in N$ は開始記号である。 P の各生成規則は $A \rightarrow \alpha$ の形をしている。ここで、 $A \in N$, $\alpha \in (N \cup \Sigma)^*$ である。

プログラミング言語の構文解析における CF 文法では、 Σ は字句解析によって得られるトークンの集合とするのが一般的である。

PL2¹ のプログラムにおける手続きの定義は以下の文法で定められている。'procedure' は予約語、'IDENT' は識別子のトークン、'(', ')', ':', および ',' は区切り記号である。

```

<proc_decl> → 'procedure' <proc_name> '(' ')' ':' <inblock>
<proc_decl> → 'procedure' <proc_name> '(' <id_list> ':' <inblock>
<proc_name> → 'IDENT'
<id_list> → 'IDENT'
<id_list> → <id_list> ',' 'IDENT'

```

<proc_decl> は識別子のトークンである名前 proc_name を持つ手続きを定義する。本体は <inblock> で記述されるプログラムで定義される。

$\beta, \gamma \in (N \cup \Sigma)^*$, $A \in N$, $A \rightarrow \alpha$ に対して、 $\beta A \gamma \Rightarrow \beta \alpha \gamma$ を P による**導出**という。導出は終端記号と非終端記号の系列に対して、生成規則の左辺の非終端記号を右辺で置き換える操作である。 $\alpha_1, \alpha_2, \dots, \alpha_n \in (N \cup \Sigma)^*$ において、 $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ のとき、 $\alpha_1 \Rightarrow^* \alpha_n$ と書くことにする。

G が定義する言語は $L(G) = \{w \in \Sigma^* \mid S \Rightarrow^* w\}$ で定義される。すなわち開始記号 S から P の生成規則を用いて導出し、すべての非終端記号がおきかえられた終端記号の系列である。

G によってプログラミング言語が定義されているとき、原始プログラムを字句解析した系列 w が $w \in L(G)$ であるとき、構文としてそのプログラムは正しいといえる。 $w \notin L(G)$ であれば、構文エラーがあると判断し、目的コードの生成は行わない。

例 4.1. 以下の PL2 プログラムは <proc_decl> から生成される系列である。ここで、 $\langle \text{inblock} \rangle \Rightarrow^* \text{'IDENT' :='IDENT' + 'NUMBER'}$ とする。

¹ 学生実験 c において対象とする原始プログラムを記述するプログラミング言語

```

procedure main(x,y):
  x:=y+1

```

BNF 定義

形式的にはプログラミング言語は文脈自由文法で定義されるが、文脈自由文法では生成規則の集合を記述すれば十分であることが多い。このため、生成規則の集合でプログラミング言語を定義する。その際に一般にBNF(Backus-Nauer Form) 定義と呼ばれる記法が用いられる。BNF は生成規則とほぼ同じであるが、 $\rightarrow$ の代わりに $::=$ を用いる。また、同じ左辺をもつ規則をまとめて、右辺を $|$ で並べて記載する。 $\langle \text{proc_decl} \rangle$ についてのBNF 定義は以下のようになる。

$$\begin{aligned}
 \langle \text{proc_decl} \rangle &::= \text{'procedure'} \langle \text{proc_name} \rangle \text{'('} \langle \text{id_list} \rangle \text{' ;' } \langle \text{inblock} \rangle \\
 &\quad | \text{'procedure'} \langle \text{proc_name} \rangle \text{'('} \langle \text{id_list} \rangle \text{' ;' } \langle \text{inblock} \rangle \\
 \langle \text{proc_name} \rangle &::= \text{'IDENT'} \\
 \langle \text{id_list} \rangle &::= \text{'IDENT'} \\
 &\quad | \langle \text{id_list} \rangle \text{' , ' } \text{'IDENT'}
 \end{aligned}$$

最左導出と最右導出

導出の途中に現れる系列 $\alpha \in (N \cup \Sigma)^*$ において、 α が複数の非終端記号を含むときどの非終端記号を置き換えて導出するかによって、複数の導出が存在する。例えば、 A, B を非終端記号に含み、 $\{a, b, c\} \subseteq \Sigma$, $A \rightarrow a$, $B \rightarrow b$ という生成規則を持つとき、 ABc に対する導出は $ABc \Rightarrow aBc$ と $ABc \Rightarrow Abc$ がある。 $\alpha \in (N \cup \Sigma)^*$ に対する導出のうち、最も左にある非終端記号を置き換える導出を**最左導出**、最も右にある非終端記号を置き換える導出を**最右導出**といい、それぞれ $\alpha \Rightarrow_{lm} \beta$, $\alpha \Rightarrow_{rm} \beta$ のように記述する。

α の $A \rightarrow \gamma$ による最左導出 $\alpha \Rightarrow_{lm} \beta$ が存在するとき、 α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_1 \in \Sigma^*$ である。すなわち、 α の A より左の α_1 は終端記号だけの系列か、 ε であり、 $\beta = \alpha_1 \gamma \alpha_2$ である。

α の $A \rightarrow \gamma$ による最右導出 $\alpha \Rightarrow_{rm} \beta$ が存在するとき、 α は $\alpha_1 A \alpha_2$ の形をしていて $\alpha_2 \in \Sigma^*$ である。すなわち、 α の A より右の α_2 は終端記号だけの系列か、 ε であり、 $\beta = \alpha_1 \gamma \alpha_2$ である。

最左導出、最右導出の0回以上の繰り返しをそれぞれ $\Rightarrow_{lm}^*$, $\Rightarrow_{rm}^*$ で表す。

例 4.2. $PL2$ の $\langle \text{proc_decl} \rangle$ からの最左導出は以下のようになる。

$$\begin{aligned}
 \langle \text{proc_decl} \rangle &\Rightarrow_{lm} \text{'procedure'} \langle \text{proc_name} \rangle \text{'('} \langle \text{id_list} \rangle \text{' ;' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure'} \text{'IDENT'} \text{'('} \langle \text{id_list} \rangle \text{' ;' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure'} \text{'IDENT'} \text{'('} \langle \text{id_list} \rangle \text{' , ' } \text{'IDENT'} \text{' ;' } \langle \text{inblock} \rangle \\
 &\Rightarrow_{lm} \text{'procedure'} \text{'IDENT'} \text{'('} \text{'IDENT'} \text{' , ' } \text{'IDENT'} \text{' ;' } \langle \text{inblock} \rangle
 \end{aligned}$$

さらに $\langle \text{inblock} \rangle \Rightarrow_{lm}^* \text{'IDENT'} \text{' := ' } \text{'IDENT'} \text{' + ' } \text{'NUMBER'}$ であるとき

$$\Rightarrow_{lm}^* \text{'procedure'} \text{'IDENT'} \text{'('} \text{'IDENT'} \text{' , ' } \text{'IDENT'} \text{' ;' } \text{'IDENT'} \text{' := ' } \text{'IDENT'} \text{' + ' } \text{'NUMBER'}$$

$PL2$ の $\langle \text{proc_decl} \rangle$ からの最右導出は以下のようになる.

$\langle \text{proc_decl} \rangle \Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 ここで $\langle \text{inblock} \rangle \Rightarrow_{rm}^* \text{'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$ であるとき
 $\Rightarrow_{rm}^* \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{' , 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$
 $\Rightarrow_{rm} \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$

開始記号 S からの 0 回以上の導出によって得られる $\alpha \in (N \cup \Sigma)^*$ (ここで, $S \Rightarrow^* \alpha$) のことを **文形式** (sentential form) という. さらに, $S \Rightarrow_{lm}^* \beta$, $S \Rightarrow_{rm}^* st\gamma$ となる $\beta, \gamma \in (N \cup \Sigma)^*$ のことをそれぞれ左文形式, 右文形式という.

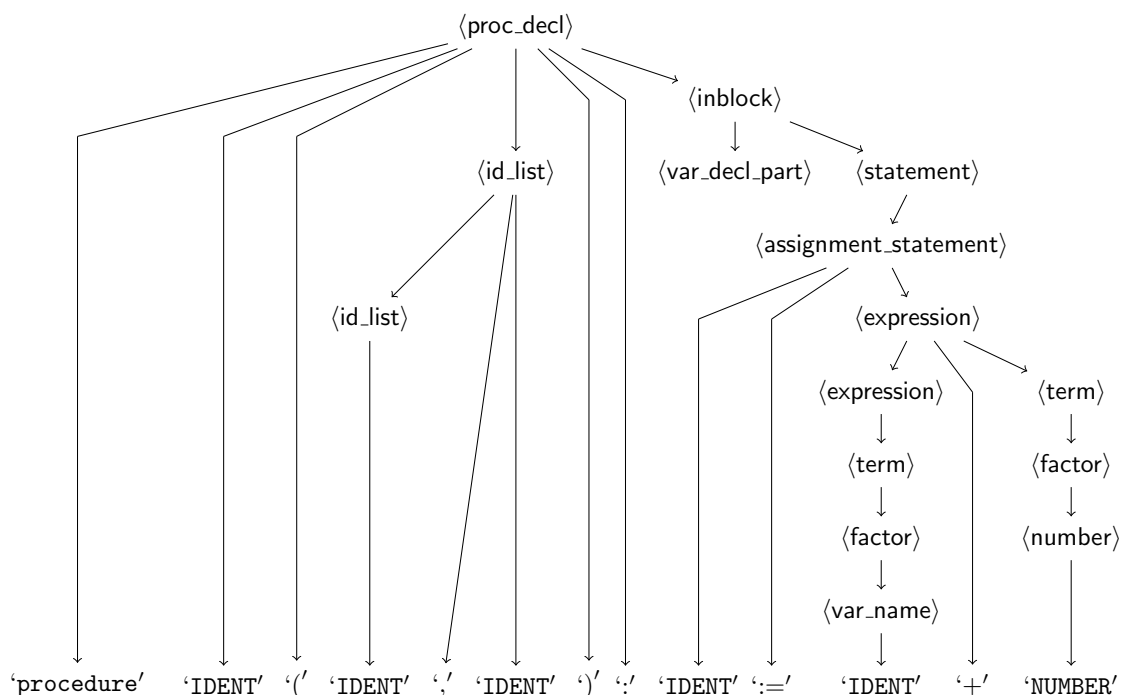
4.1.2 解析木

導出の過程は木構造で表現することができる. 非終端記号, 終端記号を頂点として表し, $A \rightarrow \alpha$ で導出したとき, A の頂点から $\alpha = \alpha_1\alpha_2\cdots\alpha_n$ ($\alpha_i \in N \cup \Sigma$) の α_i に有向辺を追加する. S からの導出をこのようにして表すと, S の頂点を根, $a \in \Sigma$ の頂点を葉とする木構造が構成される. この木構造のことを**解析木** (parse tree)(あるいは**構文木** (syntax tree)) という.

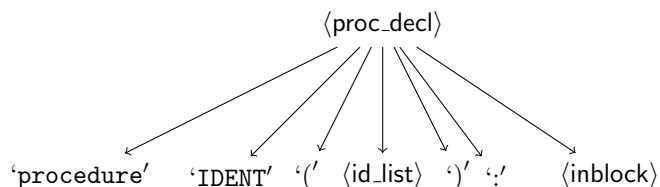
例 4.1 の例の (最左) 導出は以下のようになる.

$\langle \text{proc_decl} \rangle \Rightarrow \text{'procedure' } \langle \text{proc_name} \rangle \text{'(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' } \langle \text{id_list} \rangle \text{' , 'IDENT')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{inblock} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{var_decl_part} \rangle \langle \text{statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' } \langle \text{assignment_statement} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{expression} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{expression} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{term} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{factor} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= } \langle \text{var_name} \rangle \text{'+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{term} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{factor} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' } \langle \text{number} \rangle$
 $\Rightarrow \text{'procedure' 'IDENT' '(' 'IDENT' ',' 'IDENT')' ':' 'IDENT' ':'= 'IDENT' '+' 'NUMBER'}$

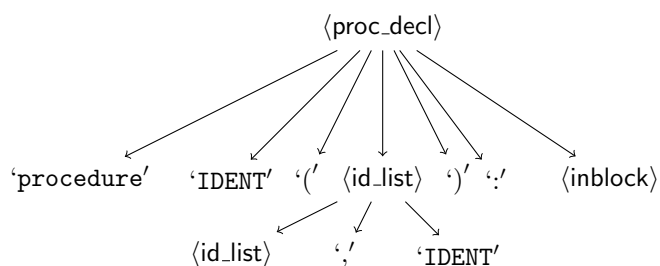
この導出から構成される木構造は以下のようになる.



解析木を構成するにあたって、最左導出では、開始記号は単一の記号なので、必ず最左の非終端記号であり、以下のような木を構成する。

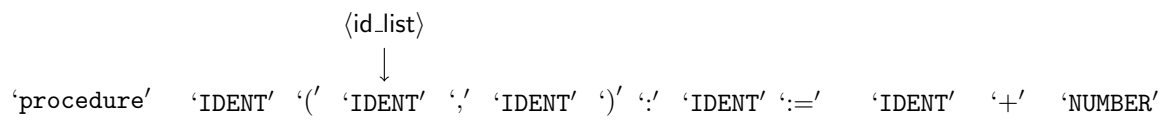


次に最左の非終端記号である <id_list> を導出すると以下の木が得られる。

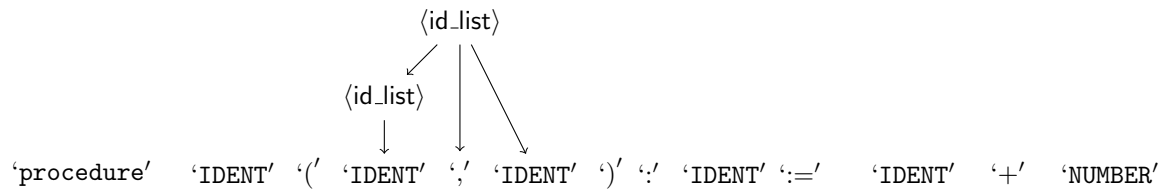


このように最左導出によって構文木は根から下向きに構成される。このような構文解析を**下降型構文解析**という。

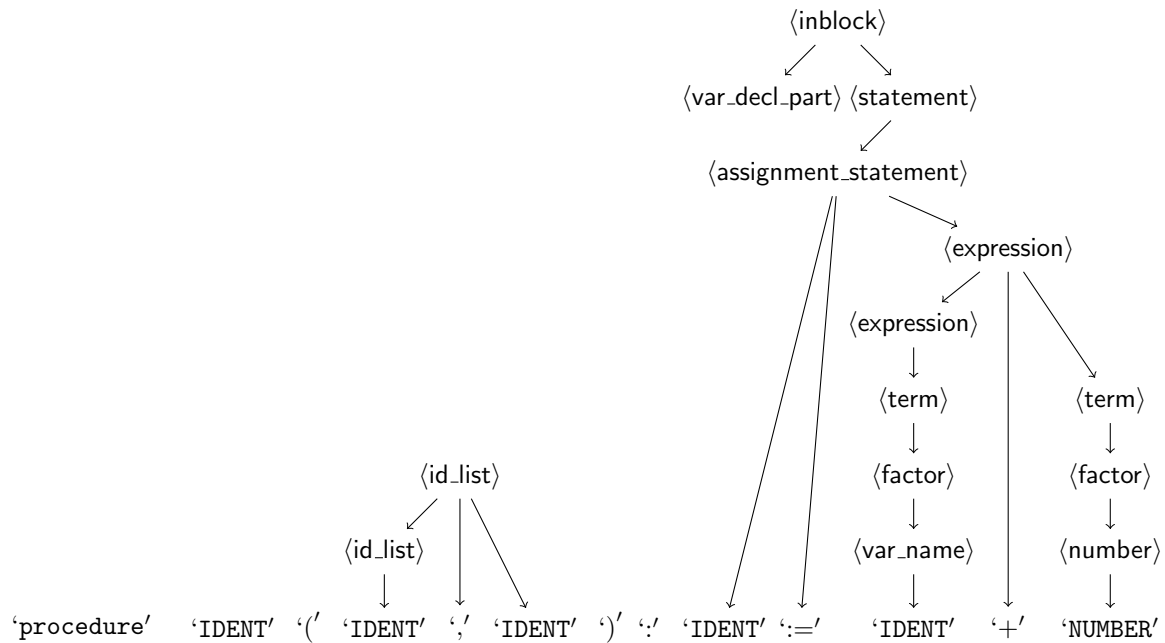
入力系列に対して引数リスト <id_list> を構成する 'IDENT' に <id_list> → 'IDENT' を逆向きに適用し、木を構成する。



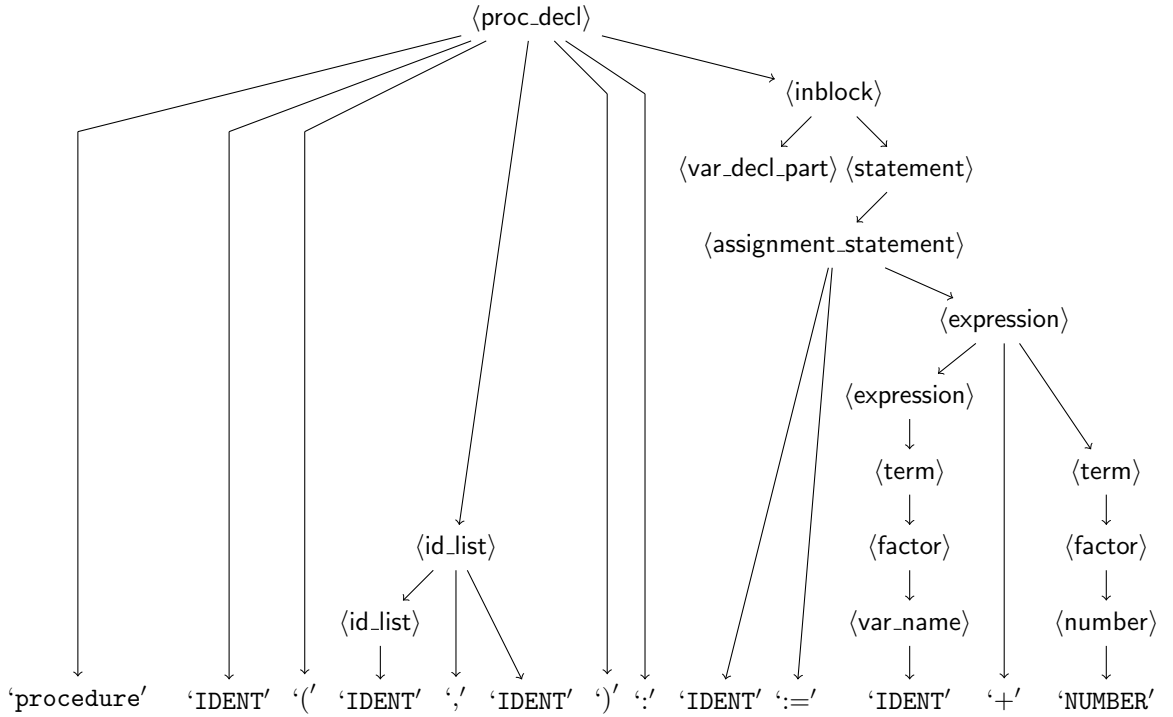
次に, $\langle \text{id_list} \rangle \rightarrow \langle \text{id_list} \rangle \text{'IDENT'}$ を逆向きに適用する.



以下, 右辺にマッチする生成規則を適用し, $\langle \text{inblock} \rangle$ の部分木を生成する.



最後に $\langle \text{proc_decl} \rangle$ を左辺に持つ規則によって木が構成される.



このように構文解析する系列に対して生成規則の右辺をマッチさせて、解析木を構成すると木は葉から根の方向に構成される。このような構文解析を上昇型構文解析という。

4.1.3 文法の曖昧性

<proc_decl> の例においては、どのような順番で生成規則を適用しても構成される解析木は同じである。文法によっては、生成規則の適用の順番によって同じ終端記号系列に対して複数の解析木が構成できることがある。

CF 文法 $G_1 = \langle \{E\}, \{\text{or}, \text{and}, \text{eqt}\}, P_1, E \rangle$ を考える。ここで、

$$P_1 = \left\{ \begin{array}{l} E \rightarrow E \text{ or } E \\ E \rightarrow E \text{ and } E \\ E \rightarrow \text{eqt} \end{array} \right\}$$

とする。eqt は等式のトークンであるとする。

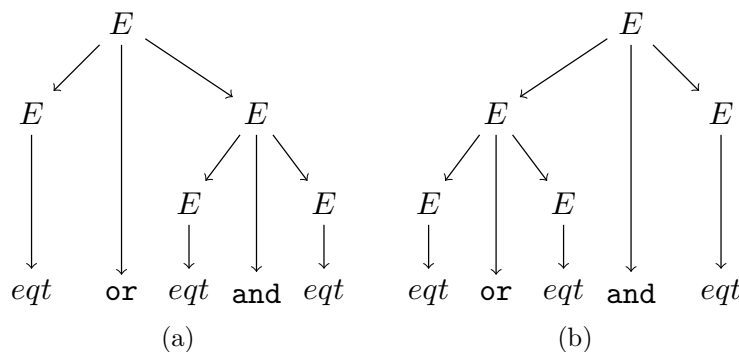


図 4.1: 複数の解析木

$L(G_1)$ に属する $\text{eqt or eqt and eqt}$ という系列に対して図 4.1 のような 2 つの解析木が存在する。導出において、 $\underline{E}$ を置き換えの対象となった E とすると、(a) は $\underline{E} \Rightarrow \underline{E} \text{ or } E \Rightarrow \text{eqt or } \underline{E} \Rightarrow \text{eqt or } \underline{E} \text{ and } E \Rightarrow \text{eqt or eqt and } \underline{E} \Rightarrow \text{eqt or eqt and eqt}$, (b) は $\underline{E} \Rightarrow \underline{E} \text{ and } E \Rightarrow \underline{E} \text{ or } E \text{ and } E \Rightarrow \text{eqt or } \underline{E} \text{ and } E \Rightarrow \text{eqt or eqt and } \underline{E} \Rightarrow \text{eqt or eqt and eqt}$, という導出に対する解析木となる。

このように 1 つのトークン系列に対して 2 つ以上の異なる解析木が存在するとき文法は曖昧であるという。

3 つの eqt のトークンが具体的にそれぞれ $x = 1, y = 2, z = 3$ であるとする。 x, y, z がすべて 1 であるとする、 $x = 1$ は真、 $y = 2, z = 3$ は偽となる。 and, or がそれぞれ論理積、論理和を表す、とすると、(a) の解釈では、最初の $\text{eqt}(x = 1)$ が真となるので、全体の論理式は真であるが、(b) の解釈では最後の $\text{eqt}(z = 3)$ が偽であるので論理式は偽となる。

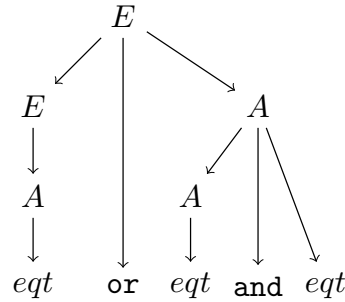
曖昧な文法では入力トークンの系列の意味が定まらないため、プログラミング言語の文法は曖昧でないことが望まれる。

論理和と論理積の場合、一般には和が積が優先されると解釈されるため、(a) の解釈をとる。これにそって G_1 の曖昧さを除去した文法 G_2 を以下に示す。

$$G_2 = \langle \{E, A\}, \{\text{or}, \text{and}, \text{eqt}\}, P_2, E \rangle$$

$$\text{ここで, } P_2 = \left\{ \begin{array}{l} E \rightarrow E \text{ or } A \\ E \rightarrow A \\ A \rightarrow A \text{ and } \text{eqt} \\ A \rightarrow \text{eqt} \end{array} \right\}$$

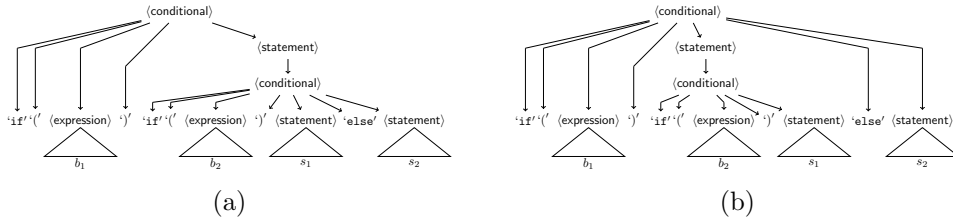
$L(G_1) = L(G_2)$ であり、 G_2 は曖昧ではない。これは、 and を or より優先するために、新たな非終端記号 A を導入して、 and が解析木で or よりも葉に近いように解釈されるようにしたためである。 G_2 では、単一生成規則 (single production) “ $E \rightarrow A$ ” を導入したため解析木のサイズが増大する。 n レベルの優先度の導入のために n 個の非終端記号が導入され、 n 個の単一生成規則が必要になる。曖昧性を除去するトレードオフとして、解析木が大きくなり、解析により長い時間がかかることになる。



このため、生成規則の適用順序を決めて解析の曖昧性を除去する場合もある。C 言語などの条件分岐は `else` つきの場合と `if ~ then ~ else ~`, `else` なしの場合 `if ~ then ~` の場合がある。

$$\begin{aligned} \langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \dots \\ \langle \text{conditional} \rangle &::= \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \\ &\quad \mid \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{'else' } \langle \text{statement} \rangle \end{aligned}$$

b_1, b_2 を $\langle \text{expression} \rangle$ から導出されるトークン列, s_1, s_2 を $\langle \text{statement} \rangle$ から導出されるトークン列とする。if (b_1) if (b_2) s_1 else s_2 に対して図??のような2つの解析木が存在する。



C 言語では、`else` は最も近い条件に対するものであると解釈するので、解析木として (b) であると決め、曖昧さを除去している。したがって、 b_1 が真で、 b_2 が偽であると評価された場合に実行される。構文解析における曖昧性は望ましいものとは言えないので、プログラム言語を定義する場合は無曖昧な CF 文法で定義することを基本とする。

上記の場合は、新たな予約語 `fi` を導入することで、曖昧さを解消できる。

$$\begin{aligned} \langle \text{statement} \rangle &::= \langle \text{conditional} \rangle \mid \dots \\ \langle \text{conditional} \rangle &::= \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{'fi' } \\ &\quad \mid \text{'if' '(' } \langle \text{expression} \rangle \text{' ' } \langle \text{statement} \rangle \text{'else' } \langle \text{statement} \rangle \text{'fi' } \end{aligned}$$

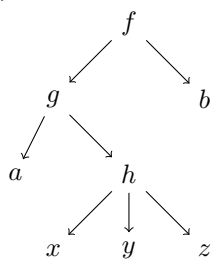
4.1.4 解析木の表現

木の頂点の出力次数がわかっているとき解析木をトラバースする方法に基づいて、木構造を1次元的に表現することができる。木 T の根頂点を $root(T)$ と書くことにする。 $root(T)$ の次数が n であるとき $root(T)$ からの子を根頂点とする木を左から $T_1, T_2, \dots, T_n$ とする。

- **プリオーダー** $root(T)$ の次に T_1 のプリオーダー系列 $pre(T_1)$ を連結し、その次に $pre(T_2)$ を連結し、というのを再帰的に T_n まで繰り返す。

- **インオーダー** T_1 のインオーダーの系列 $\text{in}(T_1)$ に $\text{root}(T)$ を連結し, その次に $\text{in}(T_2), \dots, \text{in}(T_n)$ を連結する.
- **ポストオーダー** T_1 のポストオーダー $\text{post}(T_1), \dots, \text{post}(T_n)$ を連結し, 最後に $\text{root}(T)$ を連結する.

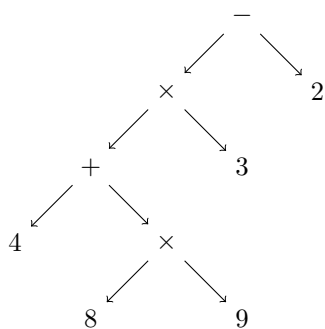
関数を表す式 $f(g(a, h(x, y, z)), b)$ に対して, 関数名とその引数を親子関係として木構造で表すと, 以下のようになる.



この木構造はプリオーダーで表すと $fgahxyzb$ となる.

f が 2 引数, g が 2 引数, h が 3 引数であることが既知であれば, この系列から関数の形を再現できる.

四則演算の算術式 $(4 + 8 \times 9) \times 3 - 2$ は以下のような構造を持つ.



これをポストオーダー系列で表すと $489 \times + 3 \times 2 -$ となる.

この記法は**逆ポーランド記法**と呼ばれる. それぞれの演算の引数が決まっていればスタックに対するルールに従って計算すれば式の評価が行われる.

逆ポーランド記法における評価の方法は以下のとおりである.

1. 数字がきたらスタックに積む
2. 演算がきたら引数の数だけスタックからポップして演算を施し, その結果をスタックにプッシュする.
3. 入力を全部読んだあとにスタックトップのあるのが評価値

$489 \times + 3 \times 2 -$ をスタックと対にしてこの方法で評価すると以下のようになる.

$$\begin{aligned}
(489 \times +3 \times 2-, []) &\rightarrow (89 \times +3 \times 2-, [4]) \\
&\rightarrow (9 \times +3 \times 2-, [4, 8]) \\
&\rightarrow (\times + 3 \times 2-, [4, 8, 9]) \\
&\rightarrow (+3 \times 2-, [4, 72]) \\
&\rightarrow (3 \times 2-, [76]) \\
&\rightarrow (\times 2-, [76, 3]) \\
&\rightarrow (2-, [228]) \\
&\rightarrow (-, [228, 2]) \\
&\rightarrow (\varepsilon, [226])
\end{aligned}$$

プリオーダーは、木の根を開始記号とする最左導出の規則適用に相当し、ポストオーダーは、木の根を開始記号とする最右導出の逆順の規則適用に相当する。